

Tutorial 3 : Sequential Decision Making under Uncertainty

National University of Singapore · School of Computing

CS4246/CS5446 Reinforcement Learning and Sequential Decision Making

Semester 1, AY2026-27 · Issued: 02 Sep 2026

Group Members

- Zhang Jiazheng (A0314707H)
- Phyo Han (A0196680R)
- Wang Shiyu (A0354696L)
- Liu Hengyan (A0350634J)
- Cui Yi (A0353244J)

AI Assistance Declaration

ChatGPT was used to help transcribe the tutorial questions into Markdown and prepare answer placeholders. I remain responsible for the accuracy, originality, and final content of this submission.

Guidelines

You may discuss the content of the questions with your classmates. But everyone should work on and be ready to present **ALL** the solutions.

Note: Transition Reward Calculation Conventions

AIMA (4e) defines the immediate reward as a transition reward $R(s, a, s')$. The reward-on-entry convention is compatible with this definition by taking $R(s, a, s') = f(s')$, that is, reward depends only on the next state. A reward-of-current-state convention corresponds to $R(s, a, s') = f(s)$, where reward depends only on the current state. Both are special cases of AIMA's general $R(s, a, s')$.

In episodic reinforcement learning, terminal states are assigned value 0, and the reward for reaching them is accounted on the transition into the terminal state. This aligns with the Sutton & Barto convention, which is the most common in reinforcement learning.

Both conventions are mathematically valid, but mixing them leads to inconsistent results. It is best to adopt one convention consistently across all parts of a problem.

In this course, by default we use the reward-on-entry convention, unless specified otherwise.

Problem 1: Nova the Mars Rover

Nova is an autonomous Mars rover that collects scientific data while managing its limited battery. Each sol, Nova must decide whether to operate in an energy-saving mode or use a high-power boost mode. Boost mode collects more data immediately but drains the battery more quickly.

Having learned about Markov Decision Processes (MDPs), Nova's controller models this decision as an infinite horizon MDP with two states—charged and low—and two actions—eco or boost. Therefore, the state set:

$$S = \{\text{charged}, \text{low}\}$$

and action set:

$$A = \{\text{eco}, \text{boost}\}$$

Nova estimates the state-transition probabilities $P(s' \mid s, a)$ as follows:

State s	Action a	$P(s' = \text{charged})$
charged	eco	0.95
charged	boost	0.7
low	eco	0.5
low	boost	0.1

This means that if Nova is charged and uses boost mode, there is a 30% chance its battery becomes low. If Nova is charged and uses eco mode, there is a 95% chance it remains charged. When its battery is low, using eco mode gives a 50% chance of returning to the charged state. However, if Nova uses boost mode while its battery is low, there is only a 10% chance it will become charged again.

Nova estimates the rewards, independent of the resulting state, as follows:

State s	Action a	Reward
charged	eco	7
charged	boost	10
low	eco	0
low	boost	2

Thus, boost mode always gives Nova a higher immediate reward than eco mode. However, Nova performs much better when its battery is charged, and using boost mode makes a low battery more likely.

The challenge is to determine what Nova should do each sol. Nova's decision-making can be modeled as an MDP with two states and two actions, giving $2 \times 2 = 4$ possible policies:

- Always use eco mode.
- Always use boost mode.
- Use eco mode when charged and boost mode when low.
- Use boost mode when charged and eco mode when low.

Without discounting, each of these policies would yield an infinite total reward over an infinite horizon.

Part 1: Discounting

a) Explain the role of discounting future rewards in MDPs. How would an agent behave differently if the discount factor was 0.6 instead of 0.9?

Answer

Discounting determines how much an agent values future rewards relative to immediate rewards. A smaller discount factor makes the agent more short-sighted, while a larger discount factor makes it care more about long-term consequences. In Nova's case, with $\gamma = 0.6$, the agent places less weight on the future cost of becoming low on battery and may therefore prefer boost more often because of its higher immediate reward. With $\gamma = 0.9$, the agent cares more about maintaining the charged state and is more willing to choose eco to obtain better rewards in the long run.

b) As the discount factor varies from 0 to 1, how does the optimal policy change?
Provide an example discount factor for each distinct policy obtainable by varying the discount factor.

Answer

Let the optimal deterministic policy be represented as

$$\pi_{\text{opt}} = \left(\arg \max_a Q(C, a), \arg \max_a Q(L, a) \right)$$

where C denotes the charged state, L denotes the low state, and the two actions are E (eco) and B (boost).

For any value function V , the four action values are

$$\begin{aligned} Q(C, B) &= 10 + \gamma(0.7V(C) + 0.3V(L)) \\ Q(C, E) &= 7 + \gamma(0.95V(C) + 0.05V(L)) \\ Q(L, B) &= 2 + \gamma(0.1V(C) + 0.9V(L)) \\ Q(L, E) &= \gamma(0.5V(C) + 0.5V(L)) \end{aligned}$$

Define

$$D = V(L) - V(C)$$

Then

$$\begin{aligned} \Delta_C &= Q(C, B) - Q(C, E) = 3 + \frac{1}{4}\gamma D \\ \Delta_L &= Q(L, B) - Q(L, E) = 2 + \frac{2}{5}\gamma D \end{aligned}$$

Thus, boost is optimal in a state when the corresponding $\Delta \geq 0$, whereas eco is optimal when $\Delta \leq 0$.

1. Policy (B, B)

Suppose boost is selected in both states. The Bellman equations are

$$\begin{cases} V(C) = 10 + \gamma(0.7V(C) + 0.3V(L)) \\ V(L) = 2 + \gamma(0.1V(C) + 0.9V(L)) \end{cases}$$

Subtracting the first equation from the second gives

$$D = -8 + \frac{3}{5}\gamma D$$

so

$$D = \frac{40}{3\gamma - 5}$$

For (B, B) to be optimal, we require

$$\begin{cases} \Delta_C \geq 0 \\ \Delta_L \geq 0 \\ 0 < \gamma < 1 \end{cases} \implies \begin{cases} 3 + \frac{10\gamma}{3\gamma - 5} \geq 0 \\ 2 + \frac{16\gamma}{3\gamma - 5} \geq 0 \\ 0 < \gamma < 1 \end{cases} \implies \gamma \in \left(0, \frac{5}{11}\right]$$

Therefore, for small discount factors, the optimal policy is (B, B) .

2. Policy (B, E)

Suppose boost is selected when charged and eco is selected when low. The Bellman equations are

$$\begin{cases} V(C) = 10 + \gamma(0.7V(C) + 0.3V(L)) \\ V(L) = \gamma(0.5V(C) + 0.5V(L)) \end{cases}$$

Subtracting gives

$$D = -10 + \frac{1}{5}\gamma D$$

so

$$D = \frac{50}{\gamma - 5}$$

For (B, E) to be optimal,

$$\begin{cases} \Delta_C \geq 0 \\ \Delta_L \leq 0 \\ 0 < \gamma < 1 \end{cases} \implies \begin{cases} 3 + \frac{25\gamma}{2(\gamma - 5)} \geq 0 \\ 2 + \frac{20\gamma}{\gamma - 5} \leq 0 \\ 0 < \gamma < 1 \end{cases} \implies \gamma \in \left[\frac{5}{11}, \frac{30}{31}\right]$$

Therefore, for intermediate discount factors, the optimal policy is (B, E) .

3. Policy (E, B)

Suppose eco is selected when charged and boost is selected when low. The Bellman equations are

$$\begin{cases} V(C) = 7 + \gamma(0.95V(C) + 0.05V(L)) \\ V(L) = 2 + \gamma(0.1V(C) + 0.9V(L)) \end{cases}$$

Subtracting gives

$$D = -5 + \frac{17}{20}\gamma D$$

so

$$D = \frac{100}{17\gamma - 20}$$

For (E, B) to be optimal,

$$\begin{cases} \Delta_C \leq 0 \\ \Delta_L \geq 0 \\ 0 < \gamma < 1 \end{cases} \implies \begin{cases} 3 + \frac{25\gamma}{17\gamma - 20} \leq 0 \\ 2 + \frac{40\gamma}{17\gamma - 20} \geq 0 \\ 0 < \gamma < 1 \end{cases} \implies \gamma \in \left[\frac{15}{19}, 1 \right) \cap \left(0, \frac{20}{37} \right] = \emptyset$$

Thus, (E, B) is not optimal for any $\gamma \in (0, 1)$.

4. Policy (E, E)

Suppose eco is selected in both states. The Bellman equations are

$$\begin{cases} V(C) = 7 + \gamma(0.95V(C) + 0.05V(L)) \\ V(L) = \gamma(0.5V(C) + 0.5V(L)) \end{cases}$$

Subtracting gives

$$D = -7 + \frac{9}{20}\gamma D$$

so

$$D = \frac{140}{9\gamma - 20}$$

For (E, E) to be optimal,

$$\begin{cases} \Delta_C \leq 0 \\ \Delta_L \leq 0 \\ 0 < \gamma < 1 \end{cases} \implies \begin{cases} 3 + \frac{35\gamma}{9\gamma - 20} \leq 0 \\ 2 + \frac{56\gamma}{9\gamma - 20} \leq 0 \\ 0 < \gamma < 1 \end{cases} \implies \gamma \in \left[\frac{30}{31}, 1 \right)$$

Therefore, for sufficiently large discount factors, the optimal policy is (E, E) .

Summary

As γ increases, the optimal policy changes as

$$(B, B) \longrightarrow (B, E) \longrightarrow (E, E).$$

More precisely,

$$\pi_{\text{opt}} = \begin{cases} (B, B), & 0 \leq \gamma < \frac{5}{11} \\ (B, E), & \frac{5}{11} < \gamma < \frac{30}{31} \\ (E, E), & \frac{30}{31} < \gamma < 1 \end{cases}$$

c) How could the MDP or the discount factor be altered so that the optimal policy is to use eco mode when charged and boost mode when low? Provide a modified MDP that requires minimal changes in probabilities, rewards, or discount factors to achieve this outcome.

Answer

We can make (E, B) optimal by changing only the reward of using boost in the charged state. Let

$$R(C, B) = R(C, E) + x = 7 + x, \quad x > 0,$$

while leaving all transition probabilities and all other rewards unchanged. Under the desired policy (E, B) , the value equations are unchanged from the analysis above:

$$\begin{cases} V(C) = 7 + \gamma(0.95V(C) + 0.05V(L)) \\ V(L) = 2 + \gamma(0.1V(C) + 0.9V(L)) \end{cases}$$

With $D = V(L) - V(C)$, this gives

$$D = -5 + \frac{17}{20}\gamma D, \quad D = \frac{100}{17\gamma - 20}.$$

For (E, B) to be optimal, eco must be preferred at C and boost must be preferred at L . Hence,

$$\begin{cases} \Delta_C = x + \frac{25\gamma}{17\gamma - 20} < 0 \\ \Delta_L = 2 + \frac{40\gamma}{17\gamma - 20} > 0 \\ 0 < \gamma < 1 \end{cases}$$

Solving the first inequality for the lower bound of γ and the second for its upper bound gives

$$\gamma > \frac{20x}{17x + 25}, \quad \gamma < \frac{20}{37}.$$

For a feasible γ to exist, its lower bound must be smaller than its upper bound:

$$\frac{20x}{17x + 25} < \frac{20}{37} \implies 0 < x < \frac{5}{4}.$$

Therefore, choose $R(C, B) \in (7, 8.25)$ and any $\gamma \in \left(\frac{20x}{17x+25}, \frac{20}{37}\right)$, the unique optimal policy is to use eco when charged and boost when low, namely (E, B) .

d) What does the discount factor γ represent in practical terms? Why can $\gamma = 1$ cause problems?

Answer

The discount factor γ controls how much future rewards are valued relative to immediate rewards. In practical terms, it determines how far into the future the agent effectively plans.

A smaller γ makes the agent more short-sighted, since future rewards are discounted heavily. A larger γ makes the agent more long-term oriented, because rewards received many steps later still contribute significantly to the current value.

The discounted return is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

When $\gamma = 1$, future rewards are not discounted at all. In an infinite-horizon continuing MDP, if positive rewards can keep occurring indefinitely, the total return may diverge:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots \rightarrow \infty.$$

As a result, the value function may not be finite, and the Bellman equations may not have a well-defined finite solution. In addition, the Bellman operator is no longer a contraction when $\gamma = 1$, so the usual convergence guarantees for value iteration and related methods may fail.

Part 2: Optimal Policy

Consider Nova's two-state MDP with $\gamma = 0.8$. Represent the value function as $[V_{\text{charged}}, V_{\text{low}}]$.

Assume the initial value function is $V^{(0)} = [0, 0]$, and initialize the Q-values as:

$$Q^{(0)}(c, \text{eco}) = 0, \quad Q^{(0)}(c, \text{boost}) = 0, \quad Q^{(0)}(l, \text{eco}) = 0, \quad Q^{(0)}(l, \text{boost}) = 0.$$

a) What are the updated Q-values starting from $V^{(0)}$?

Answer

Using the Bellman optimality backup,

$$Q^{(1)}(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^{(0)}(s').$$

Since $V^{(0)}(C) = V^{(0)}(L) = 0$, every expected future-value term is zero. Therefore,

$$\begin{aligned} Q^{(1)}(C, E) &= 7 + 0.8(0.95 \cdot 0 + 0.05 \cdot 0) = 7, \\ Q^{(1)}(C, B) &= 10 + 0.8(0.7 \cdot 0 + 0.3 \cdot 0) = 10, \\ Q^{(1)}(L, E) &= 0 + 0.8(0.5 \cdot 0 + 0.5 \cdot 0) = 0, \\ Q^{(1)}(L, B) &= 2 + 0.8(0.1 \cdot 0 + 0.9 \cdot 0) = 2. \end{aligned}$$

b) What is the next updated value function $V^{(1)}$?

Answer

The value-iteration update takes the largest Q-value in each state:

$$\begin{aligned} V^{(1)}(C) &= \max\{Q^{(1)}(C, E), Q^{(1)}(C, B)\} = \max\{7, 10\} = 10, \\ V^{(1)}(L) &= \max\{Q^{(1)}(L, E), Q^{(1)}(L, B)\} = \max\{0, 2\} = 2. \end{aligned}$$

Thus,

$$V^{(1)} = [10, 2].$$

c) Starting from $V^{(1)} = [10, 2]$, compute the next update $V^{(2)}$:

- **Value Iteration (VI):** Apply the Bellman optimality update.
- **Policy Iteration (PI):** Derive the greedy policy $\pi^{(1)}$ wrt $V^{(1)}$, then evaluate it exactly by solving the linear system for $V^{\pi^{(1)}}$.

Answer

For value iteration, first perform one Bellman optimality backup using $V^{(1)} = [10, 2]$:

$$Q(C, E) = 7 + 0.8(0.95 \cdot 10 + 0.05 \cdot 2) = 14.68,$$

$$Q(C, B) = 10 + 0.8(0.7 \cdot 10 + 0.3 \cdot 2) = 16.08,$$

$$Q(L, E) = 0 + 0.8(0.5 \cdot 10 + 0.5 \cdot 2) = 4.80,$$

$$Q(L, B) = 2 + 0.8(0.1 \cdot 10 + 0.9 \cdot 2) = 4.24.$$

Therefore,

$$V_{\text{VI}}^{(2)} = [\max\{14.68, 16.08\}, \max\{4.80, 4.24\}] = [16.08, 4.80].$$

For policy iteration, the greedy policy with respect to $V^{(1)}$ is

$$\pi^{(1)}(C) = B, \quad \pi^{(1)}(L) = E.$$

Evaluating this policy exactly gives the Bellman expectation equations

$$\begin{cases} V(C) = 10 + 0.8(0.7V(C) + 0.3V(L)), \\ V(L) = 0.8(0.5V(C) + 0.5V(L)). \end{cases}$$

Equivalently,

$$\begin{cases} 0.44V(C) - 0.24V(L) = 10, \\ -0.4V(C) + 0.6V(L) = 0, \end{cases}$$

so

$$V^{\pi^{(1)}} = \left[\frac{250}{7}, \frac{500}{21} \right] \approx [35.71, 23.81].$$

d) What is the optimal policy?

Answer

From Part 1(b), the optimal policy is (B, E) whenever

$$\frac{5}{11} < \gamma < \frac{30}{31}.$$

Since

$$\frac{5}{11} < 0.8 < \frac{30}{31},$$

the optimal policy for this MDP with $\gamma = 0.8$ is

$$\pi_{\text{opt}}(C) = B, \quad \pi_{\text{opt}}(L) = E.$$

That is, Nova should use boost when charged and eco when low.

Problem 2: Aster's Rescue Route—Comparing Active RL Methods

Aster is an autonomous rescue drone learning how to return to base through an uncertain environment. At state s_0 , it can choose between two routes:

$$a = \text{valley route}, \quad b = \text{ridge shortcut}.$$

The drone may reach an intermediate state s_1 or return directly to base T , which is terminal. At s_1 , it can choose between actions c and d .

Assume the environment model is initially unknown. The discount factor is $\gamma = 0.8$.

Before the current episode, Aster has observed the following outcomes from s_0 :

Action	Next state	Reward	Number observed
a	s_1	0	4
b	T	6	3
b	s_1	−2	1

When the previous transition under action b reached s_1 , the episode continued with action d and reward 5 on reaching T .

A new episode now begins. Aster takes action b in s_0 and observes

$$s_0 \xrightarrow{b, -2} s_1.$$

Its exploring behavior policy then selects action d , producing

$$s_1 \xrightarrow{d,5} T.$$

Consider how different active reinforcement learning methods use this experience.

(a) Active ADP

After observing the new transition $s_0 \xrightarrow{b,-2} s_1$, update the estimated transition probabilities for actions a and b at s_0 .

Suppose the current utility estimates from the learned model are

$$U(s_1) = 4, \quad U(T) = 0.$$

Using the updated model, compute the expected value of actions a and b at s_0 and determine the greedy action.

Answer

For action a , all four observed transitions lead to s_1 , so its estimated transition probabilities remain

$$\hat{P}(s_1 \mid s_0, a) = 1, \quad \hat{P}(T \mid s_0, a) = 0.$$

For action b , the new transition adds one more outcome to s_1 . Hence, among five observations of b , three lead to T and two lead to s_1 :

$$\hat{P}(T \mid s_0, b) = \frac{3}{5}, \quad \hat{P}(s_1 \mid s_0, b) = \frac{2}{5}.$$

Using one Bellman backup with $U(s_1) = 4$, $U(T) = 0$, and $\gamma = 0.8$ gives

$$\begin{aligned} \hat{Q}(s_0, a) &= 1(0 + 0.8U(s_1)) = 0.8(4) = 3.2, \\ \hat{Q}(s_0, b) &= \frac{3}{5}(0 + 0.8U(T)) + \frac{2}{5}(-2 + 0.8U(s_1)) \\ &= \frac{3}{5}(0) + \frac{2}{5}(-2 + 0.8 \times 4) = 4.08. \end{aligned}$$

Therefore,

$$U_{\text{new}}(s_0) = \max\{3.2, 4.08\} = 4.08,$$

and the greedy action is b (the ridge shortcut).

(b) Monte Carlo Control

Before the new episode, the observed complete returns following (s_0, b) were

$$6, \quad 6, \quad 6, \quad 2.$$

Compute the return following (s_0, b) in the new episode and update the Monte Carlo estimate $Q(s_0, b)$ using all five returns.

Answer

The new episode produces reward -2 after taking b at s_0 , followed by reward 5 on the transition from s_1 to T . Thus, the complete discounted return following (s_0, b) is

$$G = -2 + \gamma(5) = -2 + 0.8 \times 5 = 2.$$

Using the four previous returns together with this new return, the Monte Carlo estimate is the sample average:

$$\begin{aligned} Q_{\text{MC}}(s_0, b) &= \frac{6 + 6 + 6 + 2 + 2}{5} \\ &= \frac{22}{5} = 4.4. \end{aligned}$$

(c) SARSA and Q-learning

Consider separate SARSA and Q-learning agents with learning rate $\alpha = 0.5$ and the following current Q-values:

	First action	Second action
s_0	$Q(s_0, a) = 3$	$Q(s_0, b) = 2$
s_1	$Q(s_1, c) = 4$	$Q(s_1, d) = 1$

Both agents observe

$$s_0 \xrightarrow{b, -2} s_1,$$

and the behavior policy selects action d in s_1 .

Compute the updated value of $Q(s_0, b)$ using:

- **SARSA;**

- Q-learning.

Answer

The common update form is

$$Q(s, a) \leftarrow Q(s, a) + \alpha [\text{target} - Q(s, a)].$$

For SARSA, the target uses the action actually selected by the behavior policy at s_1 , namely d :

$$\begin{aligned}\text{target}_{\text{SARSA}} &= -2 + 0.8Q(s_1, d) = -2 + 0.8(1) = -1.2, \\ Q_{\text{SARSA}}(s_0, b) &= 2 + 0.5[-1.2 - 2] = 0.4.\end{aligned}$$

For Q-learning, the target uses the largest estimated action value at s_1 :

$$\begin{aligned}\text{target}_{\text{Q-learning}} &= -2 + 0.8 \max\{Q(s_1, c), Q(s_1, d)\} \\ &= -2 + 0.8 \max\{4, 1\} = 1.2, \\ Q_{\text{Q-learning}}(s_0, b) &= 2 + 0.5[1.2 - 2] = 1.6.\end{aligned}$$

(d)

Suppose the behavior policy had selected action c rather than d in s_1 . What would happen to the SARSA and Q-learning updates?

Answer

If the behavior policy selects c , SARSA uses $Q(s_1, c) = 4$ in its target. Therefore,

$$\begin{aligned}\text{target}_{\text{SARSA}} &= -2 + 0.8Q(s_1, c) = -2 + 0.8(4) = 1.2, \\ Q_{\text{SARSA}}(s_0, b) &= 2 + 0.5[1.2 - 2] = 1.6.\end{aligned}$$

For Q-learning, the update is unchanged:

$$\begin{aligned}\text{target}_{\text{Q-learning}} &= -2 + 0.8 \max\{4, 1\} = 1.2, \\ Q_{\text{Q-learning}}(s_0, b) &= 2 + 0.5[1.2 - 2] = 1.6.\end{aligned}$$

(e)

Compare the four active RL methods. What does each learn, and how does each use experience to improve its policy?

Answer

The four methods differ mainly in whether they learn an environment model and in the target used to update their value estimates.

- **Active ADP** learns an explicit model, namely transition probabilities and (if unknown) rewards, from observed transitions. It then plans with this model, for example by applying Bellman backups to compute action utilities, and acts greedily with respect to the resulting utilities.
- **Monte Carlo control** learns action values $Q(s, a)$ directly, without learning a transition model. After an episode terminates, it uses the complete observed return following each state-action pair to update its sample-average return estimate, then improves the policy to prefer actions with larger estimated values.
- **SARSA** also learns $Q(s, a)$ directly and model-free, but updates after every transition using the one-step target $r + \gamma Q(s', a')$. Because a' is the next action actually selected by the behavior policy, SARSA is on-policy: it learns the value of the exploratory policy it follows and improves that policy gradually.
- **Q-learning** learns $Q(s, a)$ directly and model-free using the one-step target $r + \gamma \max_{a'} Q(s', a')$. It is off-policy because the update assumes the best next action regardless of the exploratory action actually taken; it therefore learns the greedy optimal action values while behavior may remain exploratory.

(f)

Why is exploration necessary for all four methods?

Answer

Exploration lets all four methods collect data for actions and outcomes that have not yet been tried: model estimates for ADP, complete returns for Monte Carlo, and TD updates for SARSA and Q-learning. Without it, an agent may never discover an action that is better than its current greedy choice.